CastIron Media

Phase 1 Lakehouse POC

Execution Report — Resources, Pipeline Architecture & Data Locations

---

Covers: why each resource exists, how data moves end to end, what every pipeline configuration file does, and exactly where to look to see the pipeline's output at each stage, including inside the Snowflake clean room.

Environment	Detail
AWS account / region	Shared account | ap-south-1
Databricks workspace	Trial workspace, serverless compute only
Snowflake account	Trial account with Data Clean Rooms installed
Infrastructure as code	Terraform (infrastructure/) + Databricks Asset Bundles (pipelines/)
Mock data	Synthetic ad-events only — generic ad-tech fields (impression / click / conversion / viewability), no production data or client-confidential terms used anywhere in this POC

1. What Was Built, and Why

The POC proves a single thesis end to end: an ad-event can be captured the instant it happens, streamed straight into a governed lakehouse, resolved to a household identity, and then made available to a privacy-safe collaboration layer (a Snowflake Clean Room) — without the raw event ever leaving a controlled boundary. Every resource below exists to prove one link in that chain.

1.1 AWS resources

Resource	Why it exists
VPC + subnets + routing (networking)	Network isolation for every other resource. No NAT gateway — outbound traffic isn't needed, so it was left out to control cost.
Kinesis stream castiron-poc-ad-server-events	The entry point. Represents the real-time feed an ad server would publish to in production.
EFO consumer lakeflow-ingest-consumer	An Enhanced Fan-Out consumer registered on the stream, matching how a production Spark job would subscribe — proves the ingestion pattern, not just a generic poll.
S3 bucket castiron-poc-unity-catalog-storage	Backing storage for Unity Catalog (governed table files) — not used as a streaming hop in the final architecture (see Section 3).
Aurora MySQL Serverless v2	Represents an existing operational database a real client would have — the eventual target for Lakehouse Federation (see Section 6).
IAM role: castiron-poc-unity-catalog-storage-credential	Lets Databricks Unity Catalog read/write the S3 bucket above, under a locked-down trust policy scoped to this workspace only.
IAM role: castiron-poc-kinesis-service-credential	Lets Databricks read the Kinesis stream directly, via a named credential Unity Catalog manages — not a raw access key (see Section 3).
IAM user + policy: castiron-poc-serverless-kinesis-reader / castiron-poc-kinesis-access	Used only by the local mock-data scripts to publish test events onto the stream — not part of the production data path.

1.2 Databricks resources

Resource	Why it exists
Unity Catalog catalog castiron_poc, schema event_pipeline	The governed namespace every pipeline table lives in — access control, lineage and discovery all key off this.
Streaming pipeline poc-event-streaming-pipeline	A Lakeflow Declarative Pipeline (DLT) running bronze -> silver -> gold on the live event stream. Detailed in Section 3.
Job: poc-identity-resolution-job	A separate scheduled job that resolves each device to a household, simulating an identity graph a real ad platform would maintain.
Unity Catalog service credential castiron-poc-kinesis	The credential object the streaming pipeline authenticates with to read Kinesis directly — see Section 3 for why this exists.

1.3 Snowflake resources

Resource	Why it exists
Database CASTIRON_POC, schema RAW, table AUDIENCE_EVENTS	Where the resolved, pipeline-produced data lands in Snowflake — the real input to the clean room demo.
Snowflake Data Clean Rooms app	The governed collaboration layer: lets two parties join their data and see only an aggregate result, never each other's raw rows.
Clean room "CastIron Audience Room"	The actual collaboration space created for this POC, holding the linked dataset, the join template, and the published listing.

2. Pipeline Steps, End to End

The full path a single ad-event takes, in order:

- **1. Event generated** — a mock ad-server event (impression, click, conversion, or viewability) is published onto the Kinesis stream.
- **2. Direct streaming ingestion** — Databricks reads the stream continuously via Spark Structured Streaming, authenticated through the Unity Catalog service credential (no intermediate storage hop).
- **3. Bronze** — the raw parsed event is written to `castiron_poc.event_pipeline.bronze_events` exactly as received.
- **4. Silver** — duplicate events (same request_id) are dropped, producing `castiron_poc.event_pipeline.silver_corrected_events`.
- **5. Gold** — the clean, deduplicated event lands in `castiron_poc.event_pipeline.gold_events`, the table downstream consumers read from.
- **6. Identity resolution** — a separate job joins each device_id in gold-level data against a household identity graph, producing `castiron_poc.event_pipeline.identity_resolution_results`.
- **7. Export to Snowflake** — gold events are joined with their resolved household_id and loaded into Snowflake as `CASTIRON_POC.RAW.AUDIENCE_EVENTS`.
- **8. Clean room analysis** — that table is registered inside the Snowflake clean room and joined under governance; only an aggregate (counts by event type) is ever returned.

3. The Pipeline Configuration Files, Explained

The streaming pipeline is defined declaratively across two YAML files plus one Python notebook. Nothing about *how* Spark processes the data lives in YAML — YAML only says *what* resource to create and *which* values to hand it; the actual transformation logic lives in Python (Section 4).

3.1 databricks.yml — the bundle's variables

```
bundle:
name: castiron-poc

variables:
catalog:
default: castiron_poc
schema:
default: event_pipeline
kinesis_stream_name:
default: castiron-poc-ad-server-events
kinesis_region:
default: ap-south-1
kinesis_service_credential:
default: castiron-poc-kinesis

include:
- resources/*.yml
```

This is the single source of truth for every value the pipeline needs — which Unity Catalog catalog/schema to write into, which Kinesis stream to read, and which credential to authenticate with. Centralizing these as **variables** means the same pipeline definition can be redeployed against a different stream, region or environment by changing one file, without touching the pipeline logic itself. The `include` line pulls in every resource definition under `resources/` so the bundle stays modular as more pipelines are added.

3.2 resources/streaming_pipeline.yml — the pipeline resource

```
resources:
pipelines:
event_streaming_pipeline:
name: "${bundle.target}-event-streaming-pipeline"
catalog: ${var.catalog}
target: ${var.schema}
continuous: true
serverless: true
photon: true
libraries:
- notebook:
path: ../src/streaming/bronze_silver_gold_pipeline.py
configuration:
ingest.kinesis_stream_name: ${var.kinesis_stream_name}
ingest.kinesis_region: ${var.kinesis_region}
ingest.kinesis_service_credential: ${var.kinesis_service_credential}
```

What each setting means and why it's set that way:

- **catalog / target** — tells the pipeline which Unity Catalog catalog and schema its output tables belong in, pulled from the shared variables above rather than hard-coded.
- **continuous: true** — the pipeline keeps a live streaming query running against Kinesis rather than running once and stopping. This is a deliberate, cost-aware choice: Spark's Kinesis connector requires a continuous trigger to consume the stream directly (a triggered/run-once pipeline cannot

read Kinesis at all on this platform). The pipeline is started before a demo and stopped afterward to control compute cost.

- **serverless: true / photon: true** — this Databricks trial workspace only provisions serverless compute (no classic clusters are available at all), and Photon is enabled for faster SQL/Spark execution at no extra setup cost.
- **libraries -> notebook** — points at the actual transformation code (Section 4). The YAML never contains transformation logic itself; it only wires the notebook into the pipeline.
- **configuration** — the three Kinesis connection values are injected as Spark configuration properties, which the notebook reads at runtime via `spark.conf.get(...)`. This is what lets the same notebook work unmodified if the stream name or region ever changes.

3.3 resources/identity_resolution_job.yml — the identity job

Defines a serverless job (not a pipeline) running the identity-resolution notebook once per trigger, reading the sample identifiers/probes volume and writing `identity_resolution_results`. It uses the same `environments` + `serverless` pattern as the streaming pipeline, for the same reason — classic compute isn't available on this workspace.

4. The Transformation Logic (bronze_silver_gold_pipeline.py)

This is the Python notebook the YAML above points to. It defines three Lakeflow Declarative Pipeline (DLT) tables as plain Python functions, each decorated to tell the pipeline engine what kind of table to materialize and how.

```
@dlt.table(name="bronze_events")
def bronze_events():
raw = (spark.readStream.format("kinesis")
.option("streamName", stream_name)
.option("region", region)
.option("serviceCredential", service_credential)
.load())
parsed = raw.select(F.from_json(F.col("data").cast("string"), EVENT_SCHEMA)
.alias("event")).select("event.*")
return parsed.withColumn("_ingested_at", F.current_timestamp())

@dlt.table(name="silver_corrected_events")
@dlt.expect_or_drop("valid_request_id", "request_id IS NOT NULL")
def silver_corrected_events():
bronze = dlt.read_stream("bronze_events")
return bronze.withWatermark("_ingested_at", "10 minutes").dropDuplicates(["request_id"])

@dlt.table(name="gold_events")
def gold_events():
return dlt.read_stream("silver_corrected_events")
```

- **bronze_events** — opens a live streaming read directly against Kinesis using the service credential (no access keys in code), decodes each record's raw bytes as JSON into the event schema, and stamps an ingestion time.
- **silver_corrected_events** — reads bronze as a stream, and `dropDuplicates` removes any event seen twice under the same request_id within a 10-minute watermark window. The `@dlt.expect_or_drop` decorator is a data-quality gate: any row missing a request_id is silently dropped rather than breaking the pipeline.
- **gold_events** — the clean, deduplicated table every downstream consumer (identity resolution, Snowflake export) reads from.

5. How Data Reaches Snowflake, and the Clean Room

5.1 Export

gold_events is joined with identity_resolution_results on device_id (bringing in the resolved household_id), exported from Databricks, and loaded into Snowflake as a plain table:

```
CREATE TABLE CASTIRON_POC.RAW.AUDIENCE_EVENTS (
request_id STRING, event_type STRING, event_timestamp STRING,
device_id STRING, household_id STRING, ip_address STRING
);
COPY INTO CASTIRON_POC.RAW.AUDIENCE_EVENTS FROM @CASTIRON_POC.RAW.LOAD_STAGE/...;
```

5.2 Clean room registration and analysis

The table is registered as a provider dataset inside the clean room, a join/analysis template is added, the room is self-invited as its own consumer (a supported "internal testing" mode for a single-account demo), and the analysis is run:

```
CALL samooha_by_snowflake_local_db.provider.cleanroom_init('CastIron Audience Room',
'INTERNAL');
CALL samooha_by_snowflake_local_db.provider.link_datasets('CastIron Audience Room',
['CASTIRON_POC.RAW.AUDIENCE_EVENTS']);
CALL samooha_by_snowflake_local_db.consumer.run_analysis('CastIron Audience Room',
'castiron_overlap_v2',
['CASTIRON_POC.RAW.AUDIENCE_EVENTS'], ['CASTIRON_POC.RAW.AUDIENCE_EVENTS'],
OBJECT_CONSTRUCT());
```

The result is an **aggregate only** — counts of events by type — never the underlying rows. This is the entire point of a clean room: two parties (here, simulated as one account acting as both sides) can learn a joined, privacy-safe answer without either side seeing the other's raw data.

6. Where To See the Data

6.1 Bronze / Silver / Gold (Databricks)

In the Databricks workspace, open **Catalog Explorer** and navigate to:

`castiron_poc › event_pipeline › bronze_events`
`castiron_poc › event_pipeline › silver_corrected_events`
`castiron_poc › event_pipeline › gold_events`
`castiron_poc › event_pipeline › identity_resolution_results`

*Each is queryable directly in a SQL editor/notebook, e.g. SELECT * FROM castiron_poc.event_pipeline.gold_events LIMIT 100;*

To watch data arrive live: open **Workflows -> Delta Live Tables**, select *poc-event-streaming-pipeline*, and view the pipeline graph — each table shows live row counts and the most recent update while the pipeline is running.

6.2 Clean room data (Snowflake)

In Snowsight:

- **Source table**: Data ■ Databases ■ `CASTIRON_POC` ■ `RAW` ■ `AUDIENCE_EVENTS` — the raw (pre-clean-room) data that was registered.
- **The clean room itself**: Data Products ■ Clean Rooms (or Catalog ■ Apps ■ Snowflake Data Clean Rooms) ■ `CastIron Audience Room` — shows the linked dataset, the analysis template, and run history.
- **Analysis results**: results of `consumer.run_analysis(...)` are returned directly to the caller (visible in a worksheet's Results pane) and are not persisted as a queryable table by default — the governed aggregate is meant to be consumed at run time, not stored as another raw asset.

7. Summary

Every resource in this POC exists to prove one segment of a single pipeline: a mock ad-event, generated with generic, non-confidential synthetic data, streams in real time from Kinesis directly into a governed Databricks lakehouse (bronze -> silver -> gold), gets resolved to a household identity, lands in Snowflake, and is made available to a privacy-preserving clean room that returns only an aggregate result. All infrastructure is defined as code (Terraform + Databricks Asset Bundles), all configuration is externalized into YAML variables rather than hard-coded, and every step has been run end to end against live, non-simulated cloud infrastructure.

Full technical runbook with troubleshooting history, exact error messages, and fixes: POC-Validation-Log.md, in the project repository.